

Project work, part 4 - Machine Learning

Orie Kimura

updated, November 29, 2025

General

Links to public GitHub repository and Streamlitapp for the compulsory work.

<https://github.com/oriekimura123/IND320-Projectwork>

<https://oriekimura123-ind320-projectwork.streamlit.app/>

Tasks

Jupyter Notebook

- Use the Elhub API to retrieve hourly production data for all price areas using PRODUCTION_PER_GROUP_MBA_HOUR for all days and hours of the years 2022 - 2024.
 - Handle these data the same way as in part 2 of the project, appending the new data after the 2021 data, both in Cassandra (using Spark - see updated advice in the installation page if you struggle) and MongoDB.
 - Using new tables in your databases, but otherwise the same strategy, retrieve hourly consumption data for all price areas using CONSUMPTION_PER_GROUP_MBA_HOUR for all days and hours of the years 2021 - 2024.

I made functions which can handle both production data and consumption data:

fetch_elhub_data, setup_spark_session, create_cassandra_keyspace_and_table and write_to_cassandra

I also had to create a new table/data structure to fit into the new functions.

So I dropped the old production data table in Cassandra and MongoDB, and made new table.

```
In [2]: # SETUP, IMPORTS, AND CONFIGURATION

# --- Environment Setup ---
import os
from typing import List, Dict
# from datetime import date, time
from urllib.parse import quote

# Paths configuration
```

```

SPARK_HOME = "C:\\Spark\\spark-3.5.1-bin-hadoop3"
HADOOP_HOME = "C:\\Hadoop\\hadoop-3.3.1"
JAVA_HOME = "C:\\Program Files\\Microsoft\\jdk-21.0.8.9-hotspot"

# Set environment variables
os.environ.update({
    "SPARK_HOME": SPARK_HOME,
    "HADOOP_HOME": HADOOP_HOME,
    "JAVA_HOME": JAVA_HOME,
    "PATH": os.environ["PATH"] + os.pathsep + os.path.join(SPARK_HOME, "bin"),
    "PYSPARK_PYTHON": "python",
    "PYSPARK_DRIVER_PYTHON": "python",
    "PYSPARK_HADOOP_VERSION": "without",
    "SPARK_CONF_DIR": os.path.join(SPARK_HOME, "conf")
})

# Database configuration
CASSANDRA_KEYSPACE = "my_keyspace"
# Use descriptive table names for the full dataset (2021-2024)
CASSANDRA_TABLE_PRODUCTION_FULL = "production_data_2021_2024"
CASSANDRA_TABLE_CONSUMPTION_FULL = "consumption_data_2021_2024"
MONGO_DATABASE = "elhub_data"
MONGO_COLLECTION_PRODUCTION_FULL = "production_data_2021_2024"
MONGO_COLLECTION_CONSUMPTION_FULL = "consumption_data_2021_2024"

# API Configuration
BASE_URL = "https://api.elhub.no/energy-data/v0/price-areas"
DATASET_PRODUCTION = "dataset=PRODUCTION_PER_GROUP_MBA_HOUR"
DATASET_CONSUMPTION = "dataset=CONSUMPTION_PER_GROUP_MBA_HOUR"
PRICE_AREAS: List[str] = ["N01", "N02", "N03", "N04", "N05"]

# Time constants
MAX_RETRIES = 3
DELAY_SECONDS = 0.5

# Define the test range (using a small range is safer and faster for testing)
from datetime import date
START_date = date(2021, 1, 1)
END_date = date(2024, 12, 31)

# Setup Spark Session
from utils.database_interaction import setup_spark_session
spark = setup_spark_session("ElhubPipeline")
print("\nSpark Session established.")

# Setup Cassandra Connection
from cassandra.cluster import Cluster
cluster = Cluster(['127.0.0.1'])
session = cluster.connect()
print("Cassandra Cluster connected.")

# Setup MongoDB URI
import streamlit as st
try:
    MONGO_URI = st.secrets["mongo"]["uri"]
    # print(MONGO_URI)

```

```

except KeyError:
    st.error("MongoDB URI not found in Streamlit secrets. Check your .streamlit/secrets.toml")
    st.stop()

spark.conf.set("spark.mongodb.output.uri", MONGO_URI)
spark.conf.set("spark.mongodb.input.uri", MONGO_URI)

print("MongoURI set.")

```

Spark Session established.
Cassandra Cluster connected.
MongoURI set.

```

In [3]: from pyspark.sql.types import StructType, StructField, StringType, DoubleType, LongType
from pyspark.sql.functions import monotonically_increasing_id
from pyspark.sql.functions import to_timestamp
from utils.data_loaders import fetch_elhub_data
from utils.database_interaction import create_cassandra_keyspace_and_table, write_to_cassandra

def process_elhub_dataset(
    dataset_name: str,          # "CONSUMPTION" or "PRODUCTION"
    dataset_type: str,          # DATASET_CONSUMPTION
    cassandra_table: str,        # CASSANDRA_TABLE_CONSUMPTION_FULL
    mongo_collection: str,       # MONGO_COLLECTION_CONSUMPTION_FULL
    keyspace_name: str,
    spark, session
):
    print(f"STARTING DATASET: {dataset_name} ({dataset_type.split('=')[-1]})")

    # print(MONGO_URI)

    # 1. FETCH DATA
    # only pricearea, datatype, groupname, starttime, quantitykwh are fetched
    print("--Fetching from El_hub started")
    data = fetch_elhub_data(
        START_date, END_date, dataset_type, PRICE_AREAS,
        BASE_URL, MAX_RETRIES, DELAY_SECONDS
    )
    print(f"--Fetched {len(data)} records for {dataset_name}.")

    # 2. SPARK TRANSFORMATION
    # Define a simplified schema
    schema = StructType([
        StructField("pricearea", StringType(), False),
        StructField("datatype", StringType(), False),
        StructField("groupname", StringType(), True),
        StructField("starttime", StringType(), False),
        StructField("endtime", StringType(), False),
        StructField("lastupdatedtime", StringType(), False),
        StructField("quantitykwh", DoubleType(), False)
    ])

    # Create Spark DataFrame and add 'ind' primary key
    df_spark = (
        spark.createDataFrame(data, schema)
        .withColumn("ind", monotonically_increasing_id().cast(LongType()))
    )

```

```

        # Select and reorder columns to match the Cassandra schema: ind first
        .select("ind", "pricearea", "datatype", "groupname", "starttime", "endtime"
    )

    # df_spark.printSchema()

    # 3. CASSANDRA SETUP AND WRITE
    print(f"-Cassandra session started")
    create_cassandra_keyspace_and_table(session, keyspace_name, cassandra_table)
    print(f"--Writing to Cassandra table: {cassandra_table} started")
    write_to_cassandra(df_spark.write, keyspace_name, cassandra_table)
    print(f"---Data written to Cassandra table: {cassandra_table}")

    # 4. MONGODB WRITE
    print(f"-MongoDB session started")
    print(f"--Writing to Mongo DB: {mongo_collection} started")
    # df_mongo = df_spark.withColumnRenamed("ind", "_id")
    df_mongo = df_spark.select("ind", "pricearea", "datatype", "groupname", "startt
    # dups = df_mongo.groupBy("_id").count().filter("count > 1")
    # print("Duplicates:", dups.count())
    # dups.show(20, False)
    # df_mongo.filter(F.isnan("quantitykwh") | F.isNull("quantitykwh")).count()
    # df_mongo.filter(to_timestamp("starttime").isNull()).count()

    df_slow = df_mongo.repartition(10)
    df_slow.write \
        .format("mongodb") \
        .mode("overwrite") \
        .option("uri", MONGO_URI) \
        .option("database", MONGO_DATABASE) \
        .option("collection", mongo_collection) \
        .save()
    print(f"---Data written to MongoDB collection: {mongo_collection}")

    # Verification steps
    print(f"-Data stored in MongoDB ")
    df_mongo_data = (
        spark.read
        .format("mongodb")
        .option("database", MONGO_DATABASE)
        .option("collection", mongo_collection)
        .load()
    )
    print(f"--Verification: First 5 rows read back from Mongo for {dataset_name}.")
    df_mongo_data.show(5)

```

2025-11-29 18:31:42.982 No runtime found, using MemoryCacheStorageManager

```

In [4]: from pymongo.mongo_client import MongoClient
        from pymongo.server_api import ServerApi
        import streamlit as st

        mongo_uri = st.secrets["mongo"]["uri"]

        # Create a new client and connect to the server
        client = MongoClient(mongo_uri, server_api=ServerApi('1'))

```

```

# Send a ping to confirm a successful connection
try:
    client.admin.command('ping')
    print("Pinged your deployment. You successfully connected to MongoDB!")
except Exception as e:
    print(e)

# 2. Check the list of database names on the *connected* cluster
# print("Databases on the connected cluster:")
# db_names = client.list_database_names()
# print(db_names)

# # 3. Try to write a test document
# test_db = client['test_connection_db']
# test_collection = test_db['test_collection']
# test_collection.insert_one({"message": "Test successful", "cluster_id": "NEW"})
# print("Test document inserted.")

# Always close the connection in a real application or script
client.close()

```

Pinged your deployment. You successfully connected to MongoDB!

```

In [5]: # --- EXECUTION ---
# Execute for Consumption Data
try:
    process_elhub_dataset(
        dataset_name="Consumption Data",
        dataset_type=DATASET_CONSUMPTION,
        cassandra_table=CASSANDRA_TABLE_CONSUMPTION_FULL,
        mongo_collection=MONGO_COLLECTION_CONSUMPTION_FULL,
        keyspace_name=CASSANDRA_KEYSPACE,
        spark=spark, session=session
    )
except Exception as e:
    print(f"\n A fatal error occurred during the pipeline execution: {e}")

```

```

STARTING DATASET: Consumption Data (CONSUMPTION_PER_GROUP_MBA_HOUR)
-Fetching from El_hub started
--Fetched 876600 records for Consumption Data.
-Cassandra session started
--Table 'my_keyspace.consumption_data_2021_2024' dropped (if it existed).
--Keyspace 'my_keyspace' confirmed/created.
--Session set to keyspace 'my_keyspace'.
---Table 'consumption_data_2021_2024' confirmed/created.
--Writing to Cassandra table: consumption_data_2021_2024 started
---Data written to Cassandra table: consumption_data_2021_2024
-MongoDB session started
--Writing to Mongo DB: consumption_data_2021_2024 started
---Data written to MongoDB collection: consumption_data_2021_2024
-Data stored in MongoDB
--Verification: First 5 rows read back from Mongo for Consumption Data.
+---+-----+-----+-----+-----+-----+
|_id|  datatype|groupname|pricearea|quantitykwh|          starttime|
+---+-----+-----+-----+-----+-----+
| 0|Consumption|  cabin|      NO1|  177071.56|2021-01-01T00:00:...|
| 1|Consumption|  cabin|      NO1|  171335.12|2021-01-01T01:00:...|
| 2|Consumption|  cabin|      NO1|  164912.02|2021-01-01T02:00:...|
| 3|Consumption|  cabin|      NO1|  160265.77|2021-01-01T03:00:...|
| 4|Consumption|  cabin|      NO1|  159828.69|2021-01-01T04:00:...|
+---+-----+-----+-----+-----+-----+
only showing top 5 rows

```

```

In [5]: # Execute for Production Data
try:
    process_elhub_dataset(
        dataset_name="Production Data",
        dataset_type=DATASET_PRODUCTION,
        cassandra_table=CASSANDRA_TABLE_PRODUCTION_FULL,
        mongo_collection=MONGO_COLLECTION_PRODUCTION_FULL,
        keyspace_name=CASSANDRA_KEYSPACE,
        spark=spark, session=session
    )

except Exception as e:
    print(f"\n A fatal error occurred during the pipeline execution: {e}")

```

```

STARTING DATASET: Production Data (PRODUCTION_PER_GROUP_MBA_HOUR)
-Fetching from El_hub started
--Fetched 872953 records for Production Data.
-Cassandra session started
--Table 'my_keyspace.production_data_2021_2024' dropped (if it existed).
--Keyspace 'my_keyspace' confirmed/created.
--Session set to keyspace 'my_keyspace'.
---Table 'production_data_2021_2024' confirmed/created.
--Writing to Cassandra table: production_data_2021_2024 started
---Data written to Cassandra table: production_data_2021_2024
-MongoDB session started
--Writing to Mongo DB: production_data_2021_2024 started
---Data written to MongoDB collection: production_data_2021_2024
-Data stored in MongoDB
--Verification: First 5 rows read back from Mongo for Production Data.
+---+-----+-----+-----+-----+-----+
|_id| datatype|groupname|pricearea|quantitykwh|          starttime|
+---+-----+-----+-----+-----+-----+
| 0|Production|  hydro|    N01| 2507716.8|2021-01-01T00:00:...|
| 1|Production|  hydro|    N01| 2494728.0|2021-01-01T01:00:...|
| 2|Production|  hydro|    N01| 2486777.5|2021-01-01T02:00:...|
| 3|Production|  hydro|    N01| 2461176.0|2021-01-01T03:00:...|
| 4|Production|  hydro|    N01| 2466969.2|2021-01-01T04:00:...|
+---+-----+-----+-----+-----+-----+
only showing top 5 rows

```

```

In [6]: # Clean up
if 'spark' in locals():
    spark.stop()
    print("\nSpark Session closed.")

if 'session' in locals():

    session.shutdown()

    cluster.shutdown()
    print("Cassandra connections closed.")

```

Spark Session closed.
Cassandra connections closed.

Streamlit app

Refactoring

- Plotting
 - Exchanged static plots (matplotlib) with dynamic plots (Plotly)
- Structure and navigation
 - On the start page, the user must first select an area and the type of energy data to analyze (e.g., Production or Consumption). To ensure consistency across all dependent visualizations and analyses, the user is not allowed to reselect these primary parameters once chosen. A change in the selected area or data type

requires the user to explicitly click the "Reset" button located on the top of the page.

- All application modules have independent date selection.

Observation Meteorology and energy production

- After implementation, play with the controls and check if you can spot any changes incorrelations in normal conditions and in/after extreme weather events.
- Between Wednesday, January 31, and Thursday, February 1, 2024, extreme weather event Ingunn impacted Trøndelag, bringing extremely strong winds. (I believe I was at home during these days; my commuter boat was cancelled.) While the attached PDF file (Ingunn and wind.pdf) shows that wind speeds were high, the production of wind energy did not increase proportionally. This pattern was observed not only during these specific days but also in the following days. The System Wind Capacity (SWC) was zero or negative

Bonus content

I have tried to implement the following contents:

- Waiting time
 - Use progress bars, spinners, or similar to indicate work in progress.
 - Cache everything that is possible to cache.
- Error handling
 - Try to incorporate checks in the app that handle missing data connections (API and database) and NaN/missing values in the data.
 - Catch the errors and give useful feedback instead of crashing or giving cryptic errormessages.
- Forecasting
 - Add weather properties to the list of exogenous variables and download when needed.

Log Describing the Compulsory Work

Coding in Jupyter Notebook

- I used the code from previous projects, and it went relatively smoothly.
- As I mentioned in the beginning of the report, I created/modified some functions to handle both production and consumption data, and a new table/data structure to accommodate the new functions.

Coding in Streamlit app

1. Exchange matplotlib figures with dynamic plots by plotly
With the help of AI tools, I exchanged all the matplotlib figures for plotly plots in the

Jupyter Notebook environment before merging the code into the Streamlit app.

2. All new functions

I modified all new functions, including machine learning (lagged_correlation_plot), SARIMAX, map_folium_choropleth, and others. All the new functions were tested in the Jupyter Notebook environment before merging them into the Streamlit app.

3. Structure and navigation before and under coding

I thought about the necessary input and output, what the user should or should not select, the ranges the user selection covers, and the user experience (e.g., speed, intuitiveness). Since I did not have an overview and understanding of what Streamlit could provide me/us, it was challenging to determine what I could serve the users and how to tune the Streamlit app.

4. Coding, testing, debugging of the entire project

I have one start/home page, seven subpages, and twelve utility files (functions, etc.). It is easy to lose track of the overview. Small changes in one file can have big consequences in other files.

5. Access to remote MongoDB Atlas has become slow

I needed to make a new mongoDB account, upload data again, and tested the rest of tasks.

SARIMAX

- I noticed that the predictions for physical quantities (energy production, energy consumption, wind speed, etc.) included negative numbers. As these variables cannot naturally be negative, I implemented conditional code to enforce non-negativity for all forecasts, excluding temperature.
- Calculating hourly data for four years is time-consuming. Therefore, I added a selection box for the datetime index to allow the user to choose a shorter period. This focused selection enabled the creation of many interesting and informative plots.
- My function, sarimax.py, utilizes both energy data and weather data. While some of the resulting plots look acceptable, others are erratic (or unreliable). It has proven too complicated for me to tune (or optimize) the variables (model parameters, exogenous factors, and orders) effectively so that all the resulting plots are reliable and understandable.

GitLab

I pushed files to GitLab, but the Streamlit app does not work online or, if it does, it runs extremely slowly.

Brief Description of AI Usage

I used AI tools to generate initial versions of the code, to refine and customize them, and to debug errors.

Coding and Debugging

I described my requirements, and AI tools suggested initial drafts. While some of the generated code worked well, others required some modification.

Structure and navigation, testing, debugging of the entire project

This was the most challenging area in this project. When an error occurred, AI tools would tell me where and how I should modify the code. As soon as I made the change, other parts of the project would stop working. This happened repeatedly.

I used AI tools to check the overall consistency of the entire project. The tools cleaned up my code by: checking for lines defined twice, ensuring the use of `st.session_state` was consistent throughout the project, and suggesting the creation of new functions.

When I did not inform the AI tools of my own modifications, they tended to stay within their own understanding and context, without incorporating my ideas and changes. AI tools clearly require context to answer me effectively. The amount of detail regarding my ideas and activities that should be shared with the AI tools remains a key question.